



Politechnika Łódzka
Instytut Automatyki

Laboratorium

Robotów Usługowych

Instrukcja
Processing - ROS

Łódź 2023

Wstęp

Celem instrukcji jest połączenie utworzonego interfejsu dla robota z wykorzystaniem języka Processing z systemem ROS.

ROS

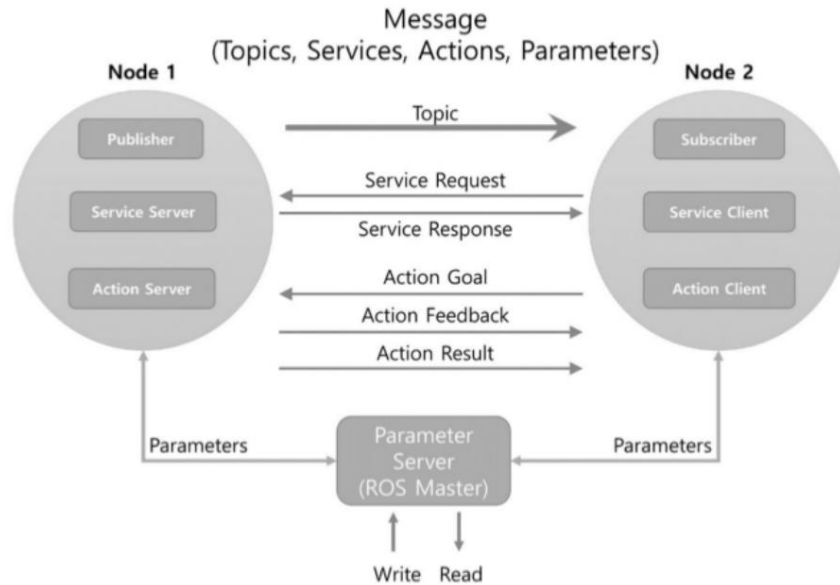
ROS (ang. Robot Operating System) jak nazwa wskazuje nie jest jednak systemem operacyjnym. Zapewnia wsparcie dla niskopoziomowych urządzeń oraz łatwą komunikację między procesami. Zaletą jest fakt, że przy tworzeniu nowego robota użytkownik najprawdopodobniej nie będzie musiał martwić się o obsługę danych czujników na niskim poziomie. Istnieje duże prawdopodobieństwo, że już ktoś inny napisał taką obsługę i ją udostępnił. Takie podejście pozwala na szybsze przejście do tworzenia bardziej zaawansowanych algorytmów bez konieczności odtwarzania "koła na nowo". Jest w pełni darmowy zarówno w celach prywatnych, naukowych jak i komercyjny co zwiększa jego popularność.

ROS Computation Graph

ROS bazuje na architekturze komunikacji peer-to-peer, tzn. każdy węzeł podłączony do sieci może jednocześnie nadawać i odbierać dane od innych węzłów. W skład Computation Graph wchodzi:

- **master** - umożliwia komunikację i wymianę informacji. Bez niego nie jest możliwe korzystanie z możliwości ROS.
- **nodes** - proces odpowiedzialny za obliczenia. W systemie robotycznym można wyróżnić np. nody odpowiedzialne za wyznaczenie lokalizacji, nawigację, obsługę kamer, obsługę pojedynczych czujników.
- **parameter server** - baza danych z parametrami. Przykłady: parametry stałe czujników, które są wykorzystywane w node'ach, dokładność dojazdu robota do punktu, opis robota.
- **messages** - są to informacje przekazywane/odbierane przez różne node'y w systemie. Składają się z podstawowych typów (np. int, float, string) jak i bardziej złożonych (np. struktura punktu ze współrzędnymi x,y). Jako przykład można podać przekazanie informacji o położeniu od node'a od lokalizacji do node'a odpowiedzialnego za nawigację.
- **topics** - jeden z nodów nadaje wiadomość na topic'u o określonej nazwie. Każdy node który chce odbierać i wykorzystywać informacje pojawiające się na nim ma taką możliwość. Przykładem może być odbieranie danych z kamery na robocie i przekazywanie ich dalej. Operator robota może odbierać dane z kamery i wyświetlać je u siebie. Jeden z nodów znajdujących się na robocie może być odpowiedzialny za wykrywanie przedmiotów na obrazie.
- **services** - jest to typ komunikacji klient-serwer. Dane przekazywane są do klienta na jego żądanie. W systemie robotycznym może być to zlecenie wykonania jakiegoś zadania przez robota i konieczne jest oczekiwanie na odpowiedź potwierdzającą wykonanie akcji.
- **bags** - służy do zapisywania i odtwarzania pojawiających się w systemie wiadomości

ROS podstawy



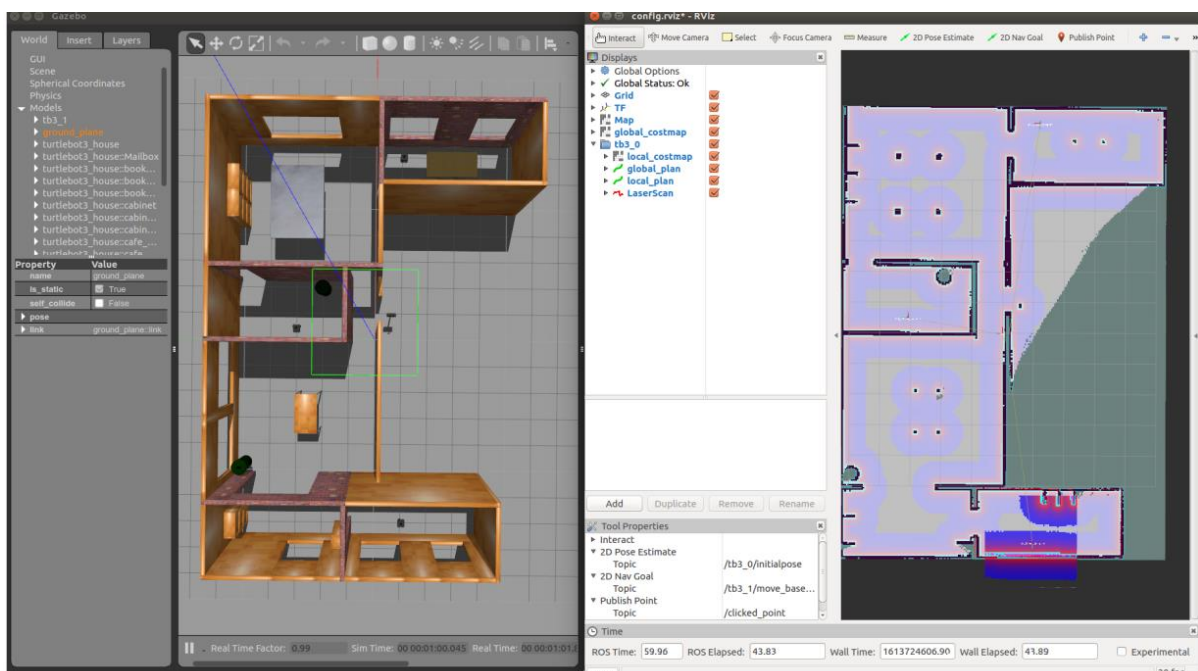
W ramach instrukcji zagadnienia zostaną ograniczone obsługi komunikacji typu Topics. ROS Topics używane są do komunikacji rozgłoszeniowej. Nie ma znaczenia kto jest nadawcą, a kto odbiorcą wiadomości. Za kodowanie przesyłanej wiadomości odpowiada **Publisher**, a za rozkodowanie **Subscriber**. Wiadomości publikowane są na topicach, które mają ściśle określone nazwy i typ wiadomości.

Podstawowe komendy ROS w terminalu

W terminalu, w którym uruchomiony jest jakiś proces (po znaku \$ nie można ustawić kursora) nie można wpisywać poleceń. Należy uruchomić nowy terminal.

1. Uruchomienie symulacji z wieloma robotami (proces blokuje terminal do wpisywania komend). Po uruchomieniu powinny pojawić się 2 okna (gazebo - świat symulacyjny, rviz – dane z czujników robotów).

roslaunch pkg_rob_usl start_sim.launch



2. Sprawdzenie listy dostępnych topiców. Po wpisaniu polecenia przy uruchomionej symulacji pojawia się lista.

```
rostopic list
```

3. Sprawdzenie typu wiadomości na topicu o podanej nazwie (np. /config/goal).

```
rostopic type /config/goal
```

W wersji uogólnionej

```
rostopic type /nazwa_topicu
```

Dla topicu “/config/goal” w rezultacie otrzymywana jest informacja o paczce i type:

```
geometry_msgs/PoseStamped
```

Otrzymana informacja o nazwie paczki (geometry_msgs) i type (PoseStamped) pozwala jednoznacznie zidentyfikować strukturę wiadomości.

4. Sprawdzenie struktury wiadomości na podstawie znanego typu wiadomości (np. geometry_msgs/PoseStamped).

```
rosmmsg show geometry_msgs/PoseStamped
```

W wersji uogólnionej

```
rosmmsg show typ_wiadomosci
```

Dla sprawdzanego typu wiadomości otrzymano następujący wynik:

```
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
geometry_msgs/Pose pose
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Quaternion orientation
    float64 x
    float64 y
    float64 z
    float64 w
```

Z lewej strony znajduje się typ pola, a z prawej nazwę pola. Typ pola może być prosty (np. bool, int8, uint16, float32, string, time) lub złożony. Typ pola złożony wykorzystuje wiadomości pochodzące z innych paczek ROS. Można go rozpoznać po następującym zapisie:

nazwa_paczki/nazwa_typu_wiadomosci_ros nazwa_pola

Dla wiadomości typu **PoseStamped**, aby dostać się do pól z informacją o położeniu x należy dostać się kolejno przez pola **pose.position.x**.

5. Do sprawdzenia wiadomości jakie przychodzą na danym topicu (np. "tb3_1/imu") i odczytu danych służy polecenie

```
rostopic echo /tb3_1/imu
```

W wersji uogólnionej

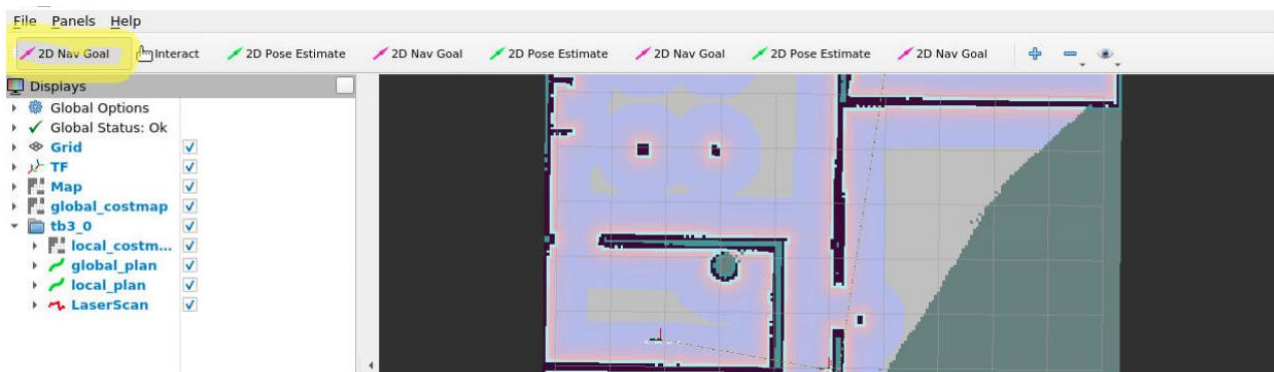
```
rostopic echo /nazwa_topicu
```

Symulacja

W symulacji dostępne są roboty o następujących nazwach: "tb3_0", "tb3_1", "tb3_2".

W oknie od RVIZ'a przy pomocy zaznaczonej strzałki można odczytać współrzędne punktu, które należy wysłać do robota, aby autonomicznie dojechał do celu. Informacje o współrzędnych publikowane są na topicu o nazwie "/config/goal". W tym celu najpierw należy uruchomić nasłuchiwanie danych przychodzących na tym topicu w terminalu, a następnie zaznaczyć strzałką punkt wraz ze wskazaniem orientacji.

Kolejne kontrolki w symulacji (*2D Pose Estimate* i *2D NavGoal*) odnoszą się kolejno do robotów o nazwach tb3_0, tb3_1 i tb3_2. Odpowiadają za wskazanie na mapie aktualnego położenia robota (*2D Pose Estimate*) i wysłania celu, do którego ma dojechać robot (*2D NavGoal*).



Przykład Processing-ROS

Zapoznać się z załączonym do instrukcji przykładem **ros_processing**. Poniżej ten sam kod programu co w pliku.

```

/*
** Basic advertise and publish from ROS
*/
import controlP5.*; //importowanie biblioteki od kontrolek
import rosprocessing.*;
ROSProcessing rp;
JSONObject json;

ControlP5 cp5;

void setup() {
    setupRos();
    size(400,600);
    noStroke();
    cp5 = new ControlP5(this);

    cp5.addButton("text1") //utworzenie kontrolki button o nazwie text1
        .setPosition(100,100) //ustawienie kontrolki na ekranie
        .setSize(200,19) //rozmiar kontrolki
        ;

    cp5.addButton("text2")
        .setPosition(100,120)
        .setSize(200,19)
        ;
    cp5.addButton("pub_ros_pose_string") // Odpowiada za opublikowanie wiadomosci typu geometry_msgs/Pose na topicu /new_button_pose
        .setPosition(100,140)
        .setSize(200,19)
        ;
    cp5.addButton("pub_ros_pose_json") // Odpowiada za opublikowanie wiadomosci typu geometry_msgs/Pose na topicu /new_button_pose
        .setPosition(100,160)
        .setSize(200,19)
        ;
}

void draw() {
    background(0);
}

void setupRos() {
    //It would be nice if every message object have a "toJson()" method.
    //For the moment we use JSONObject instead.
    try {
        println("ROSProcessing connecting");
        rp = new ROSProcessing(this, "0.0.0.0", 8080); // polaczenie po websocket z ROS
        rp.connect();
        rp.advertise("/chatter", "std_msgs/String"); // utworzenie Publishera ROS, pierwszy argument nazwa topicu, drugi typ wiadomosci
        rp.advertise("/new_button_pose", "geometry_msgs/Pose");
        rp.subscribe("/example_vector", "get_vector"); // utworzenie Subscriber'a ROS, pierwszy argument - nazwa topicu, drugi argument
                                                    // nazwa funkcji, ktora bedzie natychmiast wywolana po odebraniu wiadomosci na topicu
    } catch (Exception e) {
        println(e);
    }
}

public void text1(int theValue) { // obsluga akcji dla przycisku text1, opublikowanie wiadomosci tekstowej
    json = new JSONObject();
    json.setString("data", "text 1");
    rp.publish("/chatter", json.toString()); // opublikowanie wiadomosci do ROS, pierwszy argument nazwa topicu, drugi argument string
                                           // w postaci json z danym odpowiadajacy wysylanej wiadomosci
}

public void text2(int theValue) {
    json = new JSONObject();
    json.setString("data", "Nowy text 2");
    print(json.toString());
    rp.publish("/chatter", json.toString());
}

public void pub_ros_pose_string(int theValue) { // wysylanie bardziej skomplikowanych i zagniezdzonych wiadomosci, wyslanie jako tekst
    String msg = "{\"position\": {\"x\":0, \"y\":0, \"z\":0}, \"orientation\": {\"x\":0, \"y\":0, \"z\":0, \"w\":1}}";
    print(msg);
    rp.publish("/new_button_pose", msg);
}

```

```

public void pub_ros_pose_json(int theValue) { // j.w. tylko wykorzystanie json
    json = new JSONObject();
    JSONObject json_position = new JSONObject();
    JSONObject json_orientation = new JSONObject();
    json_position.put("x", 0);
    json_position.put("y", 0);
    json_position.put("z", 0);
    json_orientation.put("x", 0);
    json_orientation.put("y", 0);
    json_orientation.put("z", 0);
    json_orientation.put("w", 1);
    json.put("position", json_position);
    json.put("orientation", json_orientation);
    print(json.toString());
    rp.publish("/new_button_pose", json.toString());
}

void get_vector(Vector3 data){ // odbior danych na topicu /example_vector, przykładowy publisher znajduje sie w instrukcji 2_Topics
    print("x=");
    print(data.getX()); // odczyt danych z odpowiednich pol wiadomosci
    print("; y=");
    print(data.getY());
    print("; z=");
    print(data.getZ());
    print("\n");
}

```

Zadania

Do wykonania zadań należy uruchomić symulację w ROS z robotami.

Zadanie na 3

Rozbudowa interfejsu (instrukcja 1 zadanie na 3) do obsługi 1 robota o możliwość wysłania po ROS w zależności od wybranego robota na topicu **/nazwa_robota/move_base_simple/goal** współrzędnych punktu (co najmniej 1 z 3), do którego dojechać ma robot. Przy uzupełnianiu wiadomości o współrzędnych celu należy dopilnować by wartości współrzędnych odnośnie położenia i orientacji punktu, *frame_id* były dokładnie przepisane. Z każdym kolejnym wysłaniem nowego punktu wartość w polu *seq* powinna się zmieniać.

Zadanie na 4

Rozbudowa interfejsu (instrukcja 1 zadanie na 4) o możliwość wysłania do 1 z 3 robotów po ROS informacji o celu, do którego ma dojechać robot oraz podłączenie do ROS kontrolki sterujących prędkościami postępowymi i obrotowymi dla robota. Prędkości należy wysłać na topicu **/nazwa_robota/cmd_vel**. Prędkość postępową należy wysłać w wiadomości w polu **linear.x**, a obrotową na **angular.z**.

Zadanie na 5

Do tego zadania należy uruchomić dodatkowo skrypt "task_result" dołączony do instrukcji 3. Biblioteka ROSprocessing obsługuje tylko niektóre typy wiadomości ROS, dlatego do przekazywania informacji o statusach będzie wykorzystywana wiadomość typu Vector3 z biblioteki geometry_msgs. Uruchomienie skryptu:

W nowym terminalu wpisać (najlepiej otworzyć terminal w katalogu z plikiem):

jupyter notebook

Prześć do katalogu z plikiem "task_result" i go otworzyć. Przy użyciu strzałki "Run" uruchomić kolejne okna.

Należy rozbudować interfejs (instrukcja 1 zadanie na 5) o kontrolki odbierające informację na topicu **/robots_task_status**. W polach x, y, z kolejno przekazane są statusy po dojechaniu do punktu dla robotów tb3_0, tb3_1, tb3_2. Należy wyświetlać odebrany status w zależności od stanu robota.

Znaczenie statusów:

```
uint8 PENDING=0  
uint8 ACTIVE=1  
uint8 PREEMPTED=2  
uint8 SUCCEEDED=3  
uint8 ABORTED=4  
uint8 REJECTED=5  
uint8 PREEMPTING=6  
uint8 RECALLING=7  
uint8 RECALLED=8  
uint8 LOST=9
```

Do skonwertowania odebranej wiadomości i wyświetlenia tekstu po stronie interfejsu można użyć funkcji:

Double.toString(*zmienna*)